

Themes

Themes control how *Calepin* renders HTML pages, single-document HTML notebooks, and Typst notebook outputs. A theme can provide MiniJinja HTML templates, shared or local partials, CSS, JavaScript, and a Typst notebook template for PDF, SVG, PNG, and HTML output.

Choosing a theme

The default theme is `calepin`. Select a different built-in or local theme with `theme` in a website's `calepin.toml`:

```
theme = "calepin"           # the default documentation theme
theme = "academic"          # a built-in essay/blog theme
theme = "themes/my-theme"    # a local theme directory
theme = false                # no base theme: raw output
```

The same values work with `--theme` on `calepin compile` and inside a document with `#calepin.setup(theme: ...)`:

```
calepin compile paper.typ --theme calepin
```

When several theme settings are present during a compile, the command line wins, then the document, then `calepin.toml`. `calepin watch` does not have a `--theme` option; it uses the document setting when present, otherwise the website's `calepin.toml` setting, otherwise the default theme.

Structure

A theme can provide templates for website pages, single-document HTML, and Typst-level notebook rendering:

```
themes/my-theme/
  theme.toml          # optional shared partial/CSS/JS imports
  layouts/
    webpage.html      # layout for website pages
    notebook.html     # layout for a single notebook rendered to HTML
    landing.html       # optional page-specific website layout
  partials/           # theme-local MiniJinja fragments
  styles/             # theme-local CSS files
  scripts/           # theme-local JavaScript files
  notebook.typ.jinja  # optional Typst notebook template
themes/shared/        # optional local source for imported shared files
  partials/
  styles/
  scripts/
```

`layouts/webpage.html`, `layouts/notebook.html`, page-specific files in `layouts/`, and files in `partials/` use the MiniJinja template language.

Customization paths

Use the smallest customization layer that fits the change:

- Use CSS customization for colors, fonts, spacing, reading width, and small project rules.
- Use local themes when you need to edit HTML templates, JavaScript, partials, or the Typst notebook template.
- Use `theme = false` plus `styles = [...]` when you want raw Typst HTML with your own CSS instead of a bundled base theme.

See [Built-in themes](#) for the bundled theme choices.